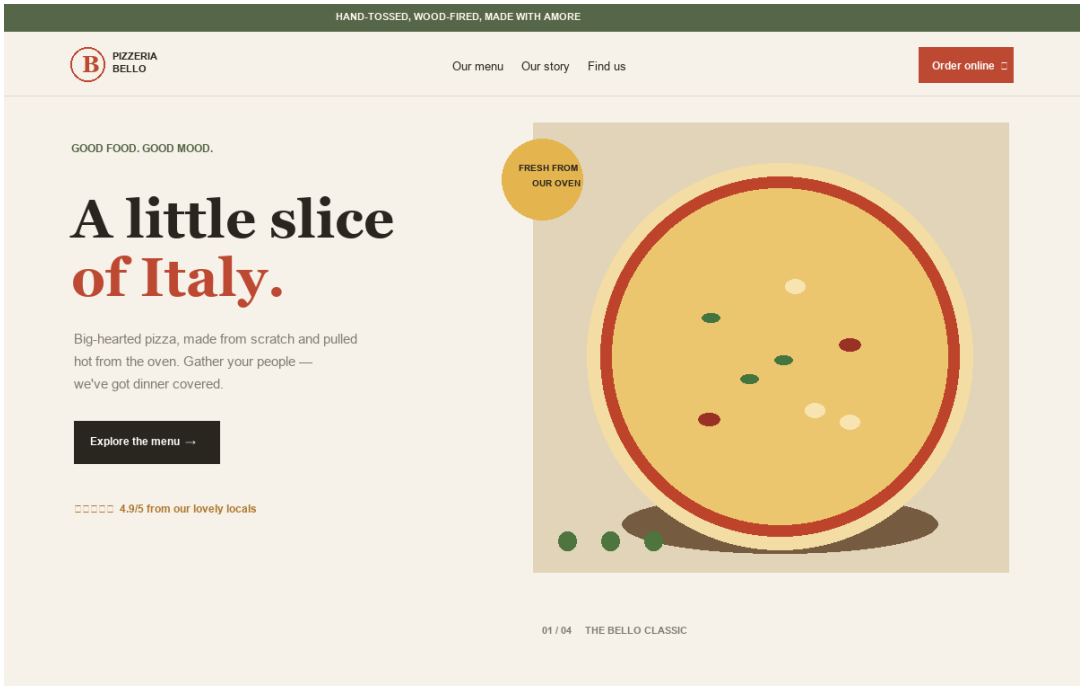
COLLEGE PROJECT REPORT

Pizzeria Bello Website Project

A responsive restaurant website concept



Submitted by: [Student Name]

Course / Department: [Course and Department]

College / University: [Institution Name]

Faculty Guide: [Name]

Academic Year: [Year]

Project submission details

Complete these fields before submission

This page is provided as an editable submission cover sheet. Replace the bracketed prompts with the details required by the college. The report describes a sample website project and does not certify that the work has been accepted, approved, or evaluated by an instructor.

Use the project title consistently across the cover, presentation slides, and any source-code submission. If the department requires a specific declaration, certificate, or formatting convention, replace the sample pages in this report with the official format supplied by the college.

The author should review the sample restaurant information and either retain it as a clearly fictional concept or substitute approved project content. Confirm that any external photographs, fonts, and icons used in the final website are permitted for the intended academic or public use.

- Student name: _____
- College / university: _____
- Department and course: _____
- Faculty guide: _____
- Academic year / submission date: _____

Student declaration

Editable declaration template

I, [Student Name], declare that this project report titled “Pizzeria Bello Website Project” has been prepared for academic submission in [Course / Department] at [College / University]. The report describes a front-end website concept developed using HTML, CSS, and JavaScript.

I understand that the restaurant identity, menu, prices, address, telephone number, and business details shown in the concept are sample content. They do not represent a verified operating restaurant. Any changes required by my instructor or institution will be made before final submission.

I have reviewed the included references and will add any additional sources, acknowledgments, or contribution statements required by the course. This page is a template and should be completed and signed only by the student whose work is being submitted.

- Student signature: _____
- Date: _____
- Place: _____

Template page: complete this declaration only if it matches your institution’s requirements.

Certificate

Supervisor or evaluator review template

This certificate page is a blank institutional template. It is not a statement that the project has been approved or evaluated. Ask the department whether it requires a specific certificate format, then replace this page with the authorized wording and signature fields.

The report describes a responsive front-end website concept for a fictional pizzeria. The reviewer may assess the project against course outcomes such as requirement analysis, system design, implementation, and documentation. Any approval date, grade, seal, or signature must be completed by the authorized person.

Do not submit this blank page as evidence of certification. The student should confirm the correct title, student details, supervisor name, and institutional format before printing or sharing the report.

- Project title: Pizzeria Bello Website Project
- Student: [Name] | Course: [Course]
- Faculty guide: [Name] | Date: [Date]
- Authorized signature: _____

Complete only with your department's approved certificate text.

Acknowledgement

A short note to personalize

I would like to thank [Faculty Guide Name] for guidance during the planning and development of this college project. Their feedback helped shape the project scope and encouraged a careful approach to layout, content, and usability.

I am grateful to [College / Department] for providing the opportunity to apply web development concepts in a practical assignment. I also appreciate the support of classmates, friends, or family members who shared informal feedback during the design process.

This acknowledgement is a draft. Before submission, replace the bracketed text with accurate names and contributions. Include only people who actually supported the project, and follow the course instructions on acknowledgements.

- Faculty guide: [Name]
- Department / institution: [Name]
- Additional contributors: [Names, if applicable]

Abstract

Project summary

Pizzeria Bello is a responsive, single-page website concept for a fictional neighborhood pizza restaurant. The project was created to demonstrate how core front-end technologies can organize restaurant information into a clear digital experience. The website introduces the brand, presents selected menu items, shares a short restaurant story, and gives visitors sample location and opening-hour details.

The page is built with semantic HTML, a custom CSS design system, and a short JavaScript interaction for expanding and collapsing the navigation on small screens. The visual direction uses a cream, red, and green palette, large editorial headings, food imagery, and responsive layouts. The design includes descriptive page metadata and accessible labels for key controls and image regions.

The project is a front-end prototype. It does not connect to a content management system, order platform, payment service, table reservation system, or database. Its restaurant details are fictional sample content. The report documents requirements, design decisions, implementation concepts, and a proposed quality-assurance plan that can be completed before a live demonstration.

- Keywords: restaurant website, responsive design, HTML, CSS, JavaScript, user experience

Contents

Required report sections and page numbers

The page numbers below correspond to this 40-page report. Replace this contents page if your college requires an automatic table of contents.

02 Project submission details
03 Student declaration
04 Certificate
05 Acknowledgement
06 Abstract
07 Contents
08 Chapter 1 — Introduction
09 Pizzeria Bello concept
10 Problem statement
11 Project objectives
12 Feasibility study
13 Project scope
14 Development method
15 Functional requirements
16 Non-functional requirements
17 Use cases and visitor journeys
18 System design overview
19 DFD context diagram
20 DFD Level 1 diagram
21 ER diagram
22 Data dictionary
23 Information architecture
24 Content strategy
25 Visual design system
26 Home page design
27 Featured menu design
28 Story and visit sections
29 Responsive layout
30 Accessibility considerations
31 Front-end architecture
32 HTML structure
33 CSS presentation
34 JavaScript interaction
35 Images and external assets
36 Privacy, security, and responsible content
37 Testing plan and test cases
38 Test results
39 Limitations and future work
40 Conclusion and references

toc

Chapter 1 — Introduction

The role of a restaurant website

A restaurant website has a practical job: it should help a visitor decide whether the restaurant fits the occasion and then make the next step obvious. A diner may arrive from a search result, a recommendation, or a social post. The page should quickly communicate cuisine, atmosphere, menu highlights, location, hours, and a way to contact the restaurant.

The Pizzeria Bello project treats the website as a simple, brand-led information experience. It is not designed to replace a restaurant's staff or its point-of-sale system. It provides the basic information that helps a visitor prepare for a visit, while a future service integration could support online ordering or bookings.

The project is a learning exercise as well as a design artifact. It connects semantic markup, responsive layout, visual hierarchy, content writing, and a small interactive component. These skills can be applied to many small-business websites beyond the restaurant example.

- Introduce the restaurant identity
- Show a small set of menu highlights
- Make location and contact information easy to scan

Pizzeria Bello concept

Brand, audience, and content assumptions

Pizzeria Bello is a fictional pizzeria with a friendly neighborhood personality. Its visual voice is informal and welcoming. The copy emphasizes fresh pizza, shared meals, and a relaxed visit. The business name, founding date, menu, location, prices, and telephone number are fictional examples used to make the concept feel complete.

The primary audience is a local diner deciding where to eat. Secondary audiences include someone planning a group meal and a first-time visitor looking for directions or hours. These audiences are assumed for the design exercise; they are not the result of a formal survey or interview study.

Before a real launch, the restaurant owner would need to approve the brand, photographs, menu descriptions, allergen information, prices, hours, and contact details. The current prototype does not validate any business fact. Clear ownership of the content is necessary to keep customer-facing information accurate.

- Brand personality: warm, generous, relaxed
- Audience: local diners and first-time visitors
- Content status: fictional and for demonstration only

Problem statement

The information gap the site addresses

A potential diner may not know what a restaurant serves, whether it has options they enjoy, or where to find it. If menu highlights and practical details are difficult to locate, visitors may abandon the decision or need to search elsewhere. A small restaurant benefits from presenting these answers in a clear and consistent format.

The project problem is therefore framed as an information-design challenge: how can a fictional neighborhood pizzeria communicate its identity, menu, and visit details in one responsive page? The solution should be easy to scan, visually distinctive, and simple to update at the file level.

This statement defines the target for the prototype rather than proving a commercial problem through research. A larger project could validate it by interviewing restaurant owners and customers, examining current discovery journeys, and comparing the information visitors need across devices.

- Challenge: present useful restaurant information with minimal friction.
- Design question: how can a one-page site guide a visitor from discovery to visit planning?

Project objectives

What the prototype aims to demonstrate

The project aims to produce a complete front-end concept that communicates the essential information of a neighborhood pizzeria. It should demonstrate semantic document structure, a consistent visual design, responsive layout behavior, and a small interaction that makes mobile navigation usable.

The objectives are deliberately concrete. Each objective can be checked by reviewing the files and observing the page. This report avoids claiming outcomes that would require a formal experiment, such as increased restaurant sales, improved conversion, or measured user satisfaction.

A student can use the objectives to structure a presentation: explain the user need, show how the site responds, describe implementation choices, and identify what would be needed to turn the prototype into a real service.

- Build a clear restaurant landing page.
- Present three featured pizzas with descriptions and sample prices.
- Provide a restaurant story, location, hours, and contact link.
- Adapt the layout and navigation for smaller screens.
- Document limitations and propose a practical next phase.

Feasibility study

Technical, operational, economic, schedule, and legal review

Technical feasibility is high for the current prototype because it uses browser-supported HTML, CSS, and JavaScript without a framework or server. The project files can be opened locally and explained separately. External images and fonts introduce an internet dependency, so local or licensed assets should be considered for a reliable public release.

Operational feasibility is suitable for a small informational website. A visitor can browse the menu and locate sample contact details. The static menu is not convenient for frequent updates, and order processing is outside the current system. A restaurant owner would need a simple content update process or a content management system if the menu changes regularly.

Economic feasibility cannot be given as a real budget because hosting, domain, asset licensing, and labor rates were not specified. For a classroom prototype, the implementation uses standard front-end files and can be demonstrated without paid backend services. Schedule feasibility is reasonable for a student project when scope remains limited.

Legal and ethical feasibility depends on replacing sample data, confirming image and font rights, and accurately describing products and allergens. A future ordering or analytics feature would require separate privacy, security, and payment review.

- Assessment: feasible as a static academic prototype.
- Condition for public use: verified business details, asset rights, hosting, and accessibility review.

Project scope

Included work and boundaries

The current scope includes a single HTML page with linked sections, a stylesheet containing desktop and mobile presentation rules, and JavaScript that toggles the mobile navigation. The page contains a hero section, featured menu cards, a story section, visit details, and a footer. The page metadata includes a title and short description.

The prototype excludes a backend, user accounts, a shopping cart, order submission, online payment, reservation management, inventory, analytics, and a content management system. It also does not provide a verified allergen database or delivery coverage information. These functions require additional product, technical, privacy, and operational decisions.

Keeping these boundaries explicit helps the reader understand what is currently demonstrated. A visible “Order online” style call to action is a design element; it is not evidence that a working order workflow exists. Any public release should adjust that link or connect it to a verified service.

- Included: static content, menu cards, anchors, mobile menu toggle
- Excluded: transactions, data storage, bookings, live menu management
- Assumption: one restaurant location and a small fixed menu

Development method

A small iterative workflow

A suitable workflow for this project begins with content and requirements, then moves through page structure, visual styling, interaction, and review. The site can be built incrementally: create the semantic sections, add the design system, check the layout at several widths, add navigation behavior, and review the content and links.

Iteration is useful because design and technical decisions affect each other. For example, a large hero image may work on desktop but consume too much vertical space on a phone. A menu label may appear clear in a desktop header yet be difficult to tap in a compact menu. Review should therefore happen throughout development rather than only at the end.

For a formal academic project, the student can keep brief notes about requirements, design changes, and review findings. The current report describes a proposed method and the structure of the supplied files; it does not claim that a documented user study or formal test campaign has been completed.

- 1. Define the audience and primary tasks.
- 2. Draft content and page sections.
- 3. Build HTML structure and CSS presentation.
- 4. Add and review mobile navigation behavior.
- 5. Review content, layout, links, and accessibility.

Functional requirements

What a visitor can do

Functional requirements describe observable behavior. The site should load its main content, provide links to the home, menu, story, and visit sections, display featured pizzas with descriptions and prices, and show a restaurant address, hours, and phone link. These are content and navigation behaviors rather than backend operations.

On smaller screens, a menu button should open and close the navigation links. The button should expose its expanded state to assistive technology and update its accessible label as the menu changes. Selecting a section link should navigate to its target and close the compact navigation.

The requirements reflect the current code structure and can be reviewed in a browser. External order processing is outside the current functionality. If a future order button is introduced, its destination and behavior should be verified before the site is presented as operational.

- FR-01: show the restaurant’s introductory hero section.
- FR-02: display featured menu items and sample prices.
- FR-03: provide section links for menu, story, and visit details.
- FR-04: show address, opening hours, and a telephone link.
- FR-05: support open and close actions for mobile navigation.

Non-functional requirements

Quality characteristics

Non-functional requirements describe qualities that shape the experience. The layout should adapt to typical phone and desktop widths, text should remain readable, and interactive elements should have meaningful names and visible focus behavior. The design should keep the restaurant's core information easy to scan.

Maintainability is supported by separating HTML, CSS, and JavaScript into named files. This separation makes it easier to find content, presentation rules, and the navigation interaction. The page should avoid unnecessary dependencies so that a student can explain how it works.

Performance and reliability depend partly on external font and photo services. If those resources are unavailable, the page should still communicate its text and retain fallback fonts. A future production version should use optimized, licensed assets and check loading behavior under slower connections.

- Usability: clear labels and predictable section links.
- Compatibility: standard browser features and CSS fallbacks.
- Maintainability: separate HTML, CSS, and JavaScript files.
- Resilience: meaningful content remains if remote assets fail.

Use cases and visitor journeys

From arrival to next step

A first-time visitor may land on the home section, read the short introduction, and select “Explore the menu.” The page moves to the featured menu section, where the visitor compares three pizzas. If interested, the visitor can continue to the story or visit details.

A returning customer may use the top navigation to move directly to the menu or visit section. On a narrow screen, the user opens the mobile menu, selects a section link, and continues reading. The navigation closes after a link is selected so it does not cover the destination content.

The visit journey ends with sample directions, opening hours, or a telephone link. In the prototype, these details are placeholders. A deployment should confirm that links point to the correct location and number, and should not imply online ordering unless that service is actually connected.

- Journey A: Home → Featured menu → Visit details → Directions or call.
- Journey B: Mobile menu → Story → Menu → Continue browsing.
- Use case boundary: the website directs visitors; it does not complete an order.

System design overview

Static information system and key flows

The system is a client-side information site. The visitor loads the page in a browser, reads static sections, and follows links to menu, story, visit details, directions, or a phone action. The browser loads the HTML document, CSS presentation, JavaScript navigation behavior, and referenced image or font assets.

There is no authenticated user, application server, database, order submission, or payment service in the current build. The source files are the content store. This design is intentionally small and matches the requirements of a project that presents restaurant information rather than managing restaurant operations.

The data-flow diagrams on the following pages describe the current boundary. A conceptual ER model is also included for a possible future content system. That model is a design proposal only; it must not be read as an implemented database.

- Primary actor: website visitor
- Core processes: render sections, navigate anchors, toggle mobile menu
- Data store: static project files
- External dependency: remote image and font resources

DFD context diagram

Level 0 view of the current system

The context diagram treats the website as one process and shows the main exchanges with its environment. A visitor requests the page and interacts with links. The site returns restaurant copy, menu information, location details, and navigation. A browser also requests remote visual assets from an external asset host.

No order data, payment details, or personal profile information flows into the project. The directions link opens an external map destination, while the telephone link delegates the call action to the visitor's device. The project itself does not store the result of either action.

The diagram represents the prototype boundary. If ordering, booking, or contact forms are added, new processes, data flows, storage, and security controls must be designed and documented.

EXTERNAL ENTITY	DATA FLOW	SYSTEM PROCESS
Visitor	Page request, link selection	Pizzeria Bello website
Pizzeria Bello website	Text, menu, hours, navigation	Visitor
Browser	Request / response for fonts and images	External asset host
Website	Directions / telephone link	Map or phone application

Boundary note: no order, payment, booking, or personal data is stored by this static prototype.

- Visitor → website: page request and section selections
- Website → visitor: restaurant information and navigation
- Browser ↔ asset host: image and font requests
- Website → map / phone app: outbound visitor action

dfd_context

DFD Level 1 diagram

Main processes and project-file data store

The level 1 view decomposes the static website into the work visible to a visitor. Process 1 renders the page sections from HTML. Process 2 handles in-page anchor navigation. Process 3 opens or closes the compact navigation on small screens. The source files supply the content and behavior.

The browser retrieves external images and fonts when the network and provider allow. These resources are presentation dependencies rather than a restaurant database. The site does not accept or persist an order, reservation, or visitor form submission.

This diagram can guide a future extension. A content system would add an editing process and persistent content store. An order system would add a cart, validation, an order service, and a payment provider, each with its own data and security requirements.

PROCESS	INPUT	OUTPUT
1.0 Render page	HTML source and visitor request	Visible restaurant sections
2.0 Navigate sections	Anchor-link selection	Menu, story, or visit section
3.0 Toggle mobile menu	Button activation	Open / closed navigation state
D1 Static files	index.html, style.css, script.js	Content, layout, interaction
External asset host	Font / image request	Remote visual assets

- 1.0 Render page sections | 2.0 Navigate anchors
- 3.0 Toggle compact menu | D1 Static HTML / CSS / JS
- External resources: hosted photography and web fonts

dfd_level1

ER diagram

Proposed content model for future expansion

A static HTML page does not currently contain a relational database. The entity relationship model below is a proposed way to organize restaurant content if the site is later connected to a content management system. It shows that one restaurant can have many menu items and many opening-hour entries, with one location and contact record in this simplified concept.

The model is intentionally small and omits order, customer, payment, and inventory entities because those functions are not in scope. If a real system is designed, the data model should be reviewed with the restaurant and normalized for the selected platform.

The examples shown in the diagram are sample values, not verified restaurant records. All business data should be approved before publication.

ENTITY	KEY FIELDS	RELATIONSHIP
Restaurant	restaurant_id (PK), name, description	One restaurant has many menu items and hours
MenuItem	item_id (PK), restaurant_id (FK), name, price	Many items belong to one restaurant
OpeningHour	hours_id (PK), restaurant_id (FK), day, open, close	Many day entries belong to one restaurant
Location	location_id (PK), restaurant_id (FK), address, map_url	One location record for this concept
Contact	contact_id (PK), restaurant_id (FK), phone, email	One contact record for this concept

- Current implementation: no database.
- Future model only: Restaurant 1—many MenuItem and OpeningHour records.

```
er
```

Data dictionary

Fields for the proposed content model

This data dictionary explains the key fields represented in the proposed content model. It is a planning aid for a future editable site, not a description of a database that exists in the current front-end project. The examples are fictional and should be replaced with owner-approved values.

Field types are conceptual. A production system may use different names or types depending on its content platform. Prices should use a currency-aware numeric representation, and opening times should be stored in a consistent time zone and format.

ENTITY.FIELD	TYPE	MEANING	SAMPLE VALUE
Restaurant.name	Text	Displayed restaurant name	Pizzeria Bello
Restaurant.description	Text	Short brand introduction	A neighborhood pizzeria
MenuItem.name	Text	Menu item title	Margherita
MenuItem.price	Decimal + currency	Displayed sample price	14.00 USD (example)
OpeningHour.day	Enum / text	Day or day range	Mon–Thu
OpeningHour.open / close	Time	Local opening interval	12:00 / 21:00
Location.address	Text	Visitor-facing location	Sample address only
Contact.phone	Telephone string	Click-to-call destination	Sample number only

- The data model separates restaurant identity, menu content, location, contact, and opening hours.

data_dictionary

Information architecture

How the one-page site is organized

The site uses a single document with section identifiers. The header contains the restaurant mark and primary navigation. The main content is ordered as hero, menu, story, and visit information. The footer closes the page with the brand and a return-to-top link. This sequence introduces the restaurant before asking visitors to act.

The information hierarchy should make the most important task easy to find. Menu and visit details are both prominent because they answer common visitor questions. The story section adds emotional context but does not interrupt access to practical information.

One-page navigation is suitable while the amount of content remains compact. If the restaurant adds a full menu, allergen guidance, catering details, or multiple locations, separate pages may become easier to navigate and maintain.



- HEADER: brand mark + primary navigation
- MAIN: hero → menu → story → visit
- FOOTER: brand statement + return-to-top link

Content strategy

What the page says and why

The page content uses short blocks that can be scanned quickly. The hero introduces the restaurant with a short message and a link to the menu. Each menu card combines a name, brief ingredient description, and sample price. The story section describes the restaurant's intended character, and the visit section contains location, hours, and a phone action.

Content should be written for clarity before decoration. Menu descriptions should use accurate ingredient names and avoid claims that cannot be verified. Prices should include the correct currency and be updated when they change. Opening hours should state exceptions where relevant, such as holidays or seasonal changes.

Photography and copy should reinforce one another. A photo should depict the food or atmosphere that the text describes. If the image is decorative, it can be treated differently from an image that conveys a specific product or detail. Alternative text should describe meaningful image content without repeating surrounding copy.

- Use specific, accurate menu descriptions.
- Keep phone, address, and hours current.
- Separate business facts from promotional language.

Visual design system

Color, type, and spacing

The visual system uses cream as the main background, dark charcoal for primary text, tomato red for emphasis, and muted green for supporting labels. A restrained palette keeps the page warm while preserving contrast between content and background. The colors are repeated across headings, buttons, small labels, and decorative elements.

Large serif headings give the brand a more editorial feel, while a sans-serif face is used for navigation and body text. The layout uses generous margins and spacing to separate sections. The design relies on hierarchy and whitespace rather than dense borders or heavy decorative framing.

Remote web fonts are included as progressive enhancement. A fallback stack should be retained so that the text remains legible if the external font service is blocked. Before a public deployment, the developer should check contrast ratios, font loading, and license terms for all selected assets.

- Primary background: warm cream
- Accent: tomato red
- Supporting color: muted green
- Typography: serif headlines and sans-serif body text

Implementation screenshots — home page

Visual preview of the implemented home section

The hero section is the first major content block. Its headline, “A little slice of Italy,” gives the restaurant an immediate identity. A short supporting paragraph explains the offer in plain language, while the menu call to action gives visitors a direct route to the featured pizzas.

The design balances text and food imagery in two columns on wide screens. The circular oven stamp and caption add a small brand detail. The hero is not intended to carry every business detail; it introduces the restaurant and gives the visitor a reason to continue.

On narrower screens, the columns stack so the text remains readable and the image remains visible. The small-screen layout should be reviewed to confirm that the call to action remains near the message and that the image does not push all useful content too far below the fold.

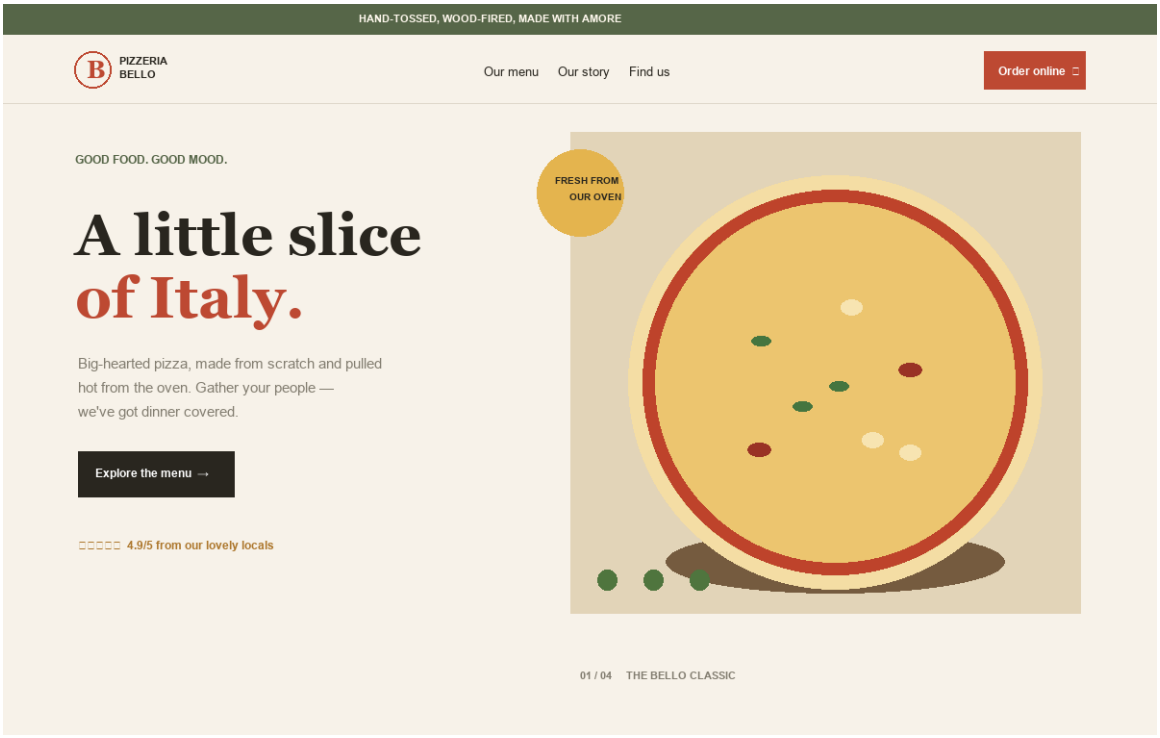


Figure 1. Home-page design preview

- Figure 1. Home-page design preview
- The visual is an illustrative reconstruction of the page layout.

Implementation screenshots — menu

Visual preview of the featured menu section

The featured menu section presents three sample pizzas as consistent cards. Each card repeats the same content pattern: image, category tag, item name, ingredient description, and price. This repetition helps visitors compare choices without learning a new layout for each item.

The cards use short descriptions to fit the one-page concept. A production menu may require additional details, such as size options, allergens, dietary markers, or full ingredient lists. These details should be accurate and should not be inferred from a food photograph.

The menu section includes a link to a fuller menu, but the current project does not provide an external menu service. A final demonstration should make this clear or connect the link to an actual page. Sample prices in the project are illustrative and must be replaced before publication.

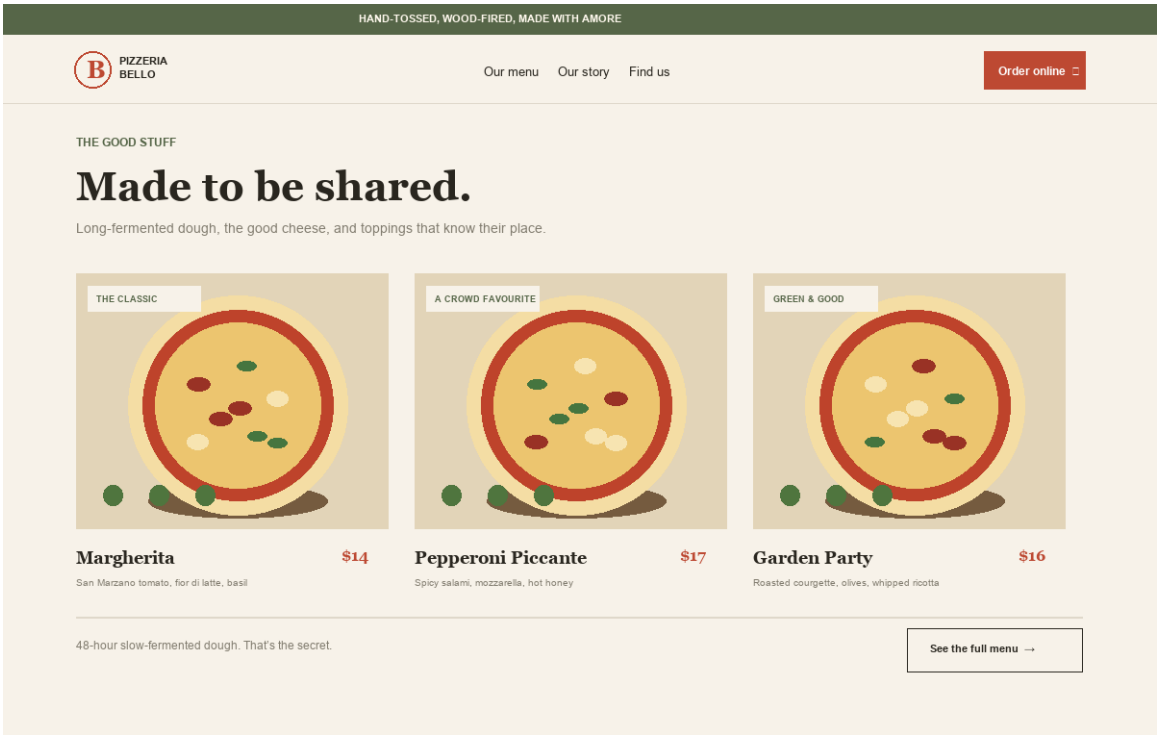


Figure 2. Featured menu design preview

- Figure 2. Featured menu design preview
- The three items and prices are sample project content.

Story and visit sections

Trust and practical information

The story section adds a human voice to the brand. It describes a simple idea—good food shared with other people—rather than making unsupported claims about history or ingredients. The sample origin story should be replaced with accurate information if the concept represents a real business.

The visit section answers practical questions: where the restaurant is, when it is open, and how to call. The address and phone number in the sample are placeholders. In a real website, the directions link should be checked against the correct map pin, and the telephone link should contain the correct country and area code.

Contact information should be easy to find and easy to use. The page uses clear labels and places the details together. A future version could provide a contact form, but it would need validation, spam handling, privacy information, and an appropriate destination for submissions.

- Story: introduce the restaurant’s voice.
- Visit: show location, hours, and call action.
- Footer: reinforce the brand and offer a return path.

Responsive layout

Adapting to screen width

The desktop design places the hero content beside the main image, presents menu cards in three columns, and groups visit information into two columns. At smaller widths, the page changes to a single-column flow, menu cards stack vertically, and the visit details become easier to read without horizontal scrolling.

Responsive behavior is implemented with CSS media queries. The current breakpoint is an implementation choice rather than a universal rule. It should be reviewed against the actual content and tested at intermediate widths, where navigation, card widths, or headings may wrap unexpectedly.

The mobile navigation is hidden until the visitor opens it. Its state should remain clear, and menu links should be large enough to tap comfortably. Images should crop intentionally and preserve the focal point. A future optimization could provide responsive image sizes so the phone does not download a larger image than necessary.

- Wide: two-column hero and three menu cards.
- Narrow: stacked hero and single-column menu.
- Review: narrow phones, larger phones, tablets, and desktop widths.

Accessibility considerations

Designing for more visitors

Accessibility should be part of the design and review process. The site includes a descriptive document title, a viewport declaration, semantic page landmarks, and an accessible label for the mobile menu button. The button exposes its expanded state so that assistive technology can communicate whether the compact menu is open.

A complete accessibility review should also examine keyboard access, focus visibility, contrast, heading order, link purpose, image alternatives, and screen-reader behavior. The presence of a label alone does not establish that the whole page conforms to an accessibility standard.

The project can use WCAG 2.2 as a reference for organizing an accessibility review. Conformance requires evaluating the full page and relevant responsive variations, not merely selecting a few techniques. The student should describe any review as a check performed, planned, or not yet performed with appropriate accuracy.

- Review keyboard navigation and focus visibility.
- Provide useful alternative text for meaningful images.
- Check contrast and link names.
- Reference: W3C Web Content Accessibility Guidelines 2.2.

Front-end architecture

A simple static site

The implementation uses a browser-only architecture. The browser reads the HTML document, loads the stylesheet and JavaScript file, then requests remote image and font assets referenced by the page. There is no application server, database, order API, or payment processor in the current version.

This architecture is appropriate for demonstrating a compact informational page. It is easy to open locally and makes the project files understandable to a beginner. The tradeoff is that changing menu content requires editing the source files, and dynamic business functions are not available.

If the site becomes a real restaurant service, the architecture should be reconsidered. A content management system could make menu updates easier, while an ordering provider could handle transaction workflows. The student should choose integrations only after confirming business requirements and privacy responsibilities.



- Visitor browser → index.html
- index.html → style.css + script.js
- Browser → external photo and font URLs
- No server-side storage or transaction flow

HTML structure

Meaningful page regions

The HTML file defines the document language, character encoding, viewport metadata, page description, and title. The body begins with an announcement strip and header. The header contains a brand link, compact menu control, and primary navigation. Main content is divided into labeled sections using stable identifiers that match the navigation links.

Semantic elements such as header, nav, main, section, article, and footer express the role of each region. Headings provide a reading hierarchy. Menu cards use article-like groupings, while the visit details provide contact information. These structural choices help both readers and assistive technologies understand the page organization.

The markup should be checked for matching IDs, unique section targets, valid nesting, and descriptive link text. If the content grows, a table of contents or additional pages may make it easier to find information than extending a single document indefinitely.

- Document metadata: title, description, viewport, language
- Landmarks: header, navigation, main, footer
- Sections: home, menu, story, visit

CSS presentation

Reusable rules and responsive styles

The stylesheet defines design tokens as CSS custom properties for colors and font families. These tokens make the palette easier to maintain because a shared color can be adjusted in one place. Global rules apply box sizing and set the base font and background, while component rules style the header, buttons, cards, story panel, visit details, and footer.

The desktop layout uses CSS Grid and Flexbox to arrange content. Media queries adjust the number of columns, spacing, and navigation behavior on narrow screens. Images are assigned through background-image rules and cropped with cover behavior. This is visually convenient but may require additional accessible text treatment when the image carries meaning.

A future refinement could move informative images into HTML image elements with responsive sources and explicit alternative text. CSS should also be reviewed for focus styles, reduced motion preferences, and readable fallback behavior.

- Tokens: palette, body font, display font
- Layout: grid and flex patterns
- Responsive rule: stack content and compact navigation

JavaScript interaction

The mobile navigation toggle

The JavaScript file selects the navigation panel and menu button, then listens for a click. On each click, it toggles the open class, updates the button's aria-expanded state, changes its accessible label, and swaps the icon between a menu mark and a close mark.

When a visitor selects a navigation link, a second event handler closes the mobile menu and resets the button state. This gives the visitor a predictable interaction on small screens. The rest of the page is static HTML and CSS; JavaScript does not fetch menu data or submit an order.

The interaction can be improved through progressive enhancement and keyboard review. The menu should remain understandable if scripts fail, and the button should behave consistently with keyboard activation. If more interactions are added, state changes should be documented and tested for assistive technology.

- State: closed → open → closed
- Accessibility state: aria-expanded and updated label
- Scope: navigation only; no order or payment behavior

Images and external assets

Photography, fonts, and loading

The website references photographs hosted by an external image service and web fonts hosted externally. This keeps the project files small, but it means that the appearance can change if a remote asset is moved, blocked, or unavailable. The page should remain understandable even when images or fonts fail to load.

Before publishing, the project owner should confirm the license and permitted use for each photograph and font. Descriptive alternative text should be supplied for meaningful images, while decorative images should not create redundant announcements. If background images remain in use, a text alternative should be available where needed.

Performance improvements may include choosing appropriately sized images, compressing files, using modern formats, and loading below-the-fold images lazily. The project should avoid downloading large full-resolution images when a smaller display size would be sufficient.

- Confirm licensing and attribution needs.
- Add useful alternatives for informative visuals.
- Consider local assets and responsive image sizing.

Privacy, security, and responsible content

Prototype boundaries

The current website does not collect or store visitor information. It does not include accounts, forms, payment data, analytics, or order history. This reduces the privacy surface of the prototype, but it does not remove the need to review externally loaded resources or future integrations.

The sample contact details and location are fictional placeholders for a project concept. They should not be presented as the details of an operating restaurant. Menu content should be checked for accuracy, especially ingredient and allergen statements. Photographs should not imply a product that the restaurant does not actually serve.

If a future version adds forms, tracking, or online checkout, the owner should identify what data is collected, why it is needed, how long it is retained, and who can access it. Payment processing should be delegated to an appropriate provider rather than implemented as a student-built form that stores card details.

- Current state: no visitor-data collection or payment flow.
- Future data features require privacy review and clear notice.
- Verify content, image rights, and business details before launch.

Testing plan and test cases

Checks defined for the requested project report

The test plan covers page structure, link targets, responsive metadata, and the mobile navigation behavior. Structural checks and a small mocked interaction harness were run against the supplied project files. The results are summarized on the next page. These checks do not replace a visual review in a real browser.

The test cases below are repeatable. A student should also open the page in a browser and review layout at phone, tablet, and desktop widths, use the keyboard to reach links and buttons, and confirm that remote images load. Those browser checks are not represented as completed here.

ID	TEST CASE	EXPECTED RESULT	CHECK METHOD
TC-01	Section IDs are unique	No repeated IDs	Static HTML check
TC-02	In-page links target existing IDs	All targets exist	Static HTML check
TC-03	Viewport metadata is present	Mobile viewport declared	Static HTML check
TC-04	Document title is present	Non-empty title tag	Static HTML check
TC-05	Open mobile menu	Open state and expanded=true	JS interaction harness
TC-06	Open state label/icon	Close label and × icon	JS interaction harness
TC-07	Select menu link	Menu closes and state=false	JS interaction harness
TC-08	Closed state label/icon	Open label and ☰ icon	JS interaction harness

test_cases

Test results

Executed checks and remaining review items

Eight automated checks passed in a local Node.js harness. Four checks inspected the HTML for unique section identifiers, valid in-page link targets, viewport metadata, and a document title. Four checks exercised the mobile navigation handler with a lightweight DOM mock: opening the menu, announcing the expanded state, closing after link selection, and restoring the closed-state label and icon.

These results apply only to the checks listed below. They do not establish full browser compatibility, visual correctness, accessibility conformance, or a working ordering service. Manual review in a browser remains necessary before a final classroom demonstration.

ID	RESULT	OBSERVATION
TC-01	PASS	Section identifiers were unique.
TC-02	PASS	All in-page targets were found.
TC-03	PASS	Viewport metadata exists.
TC-04	PASS	Document title exists.
TC-05	PASS	Menu opens and exposes expanded state.
TC-06	PASS	Open label and icon are updated.
TC-07	PASS	Section-link selection closes the menu.
TC-08	PASS	Closed label and icon are restored.

- Result: 8 checks passed, 0 failed.
- Pending: visual review in browser at multiple widths; keyboard and assistive-technology review; link destination confirmation.

test_results

Limitations and future work

What should happen next

The prototype has several limitations. It uses fixed sample content, a single page, remote image and font services, and a static menu. It does not support online orders, table booking, menu editing, multiple languages, dietary filtering, or content updates without editing files. The sample location and contact details must not be used as real business details.

A practical next step is to validate the content and visual direction with the restaurant owner or course reviewer. The student can then review the page at multiple widths, check keyboard behavior, optimize imagery, and replace sample data. If the site will be public, hosting, a domain, asset licensing, and accessibility review should be planned.

Further features should be prioritized based on actual need. A complete online ordering experience is a separate project with operational, privacy, and payment requirements. It should not be represented as a minor button change.

- Near term: verify content, links, and responsive presentation.
- Later: consider CMS, full menu, multilingual support, or ordering integration.
- Before launch: confirm ownership, hosting, privacy, and accessibility needs.

Conclusion and references

Summary and learning sources

Pizzeria Bello demonstrates a focused way to present a fictional restaurant online. Its one-page structure introduces the brand, highlights the menu, shares a story, and surfaces visit details. The implementation uses HTML, CSS, and JavaScript to create a responsive front-end concept with a mobile navigation interaction.

The project should be presented as a prototype rather than a live ordering service. Its details are sample content, its visual previews are illustrative, and its quality-assurance steps remain proposed until the student performs and records them. Clear communication about these boundaries strengthens the academic report.

The following sources provide foundational guidance for the web technologies and accessibility concepts discussed in this report. The student may add course materials and any additional sources actually consulted, following the citation style required by the institution.

- [1] MDN Web Docs, Web developer guides: https://developer.mozilla.org/en-US/docs/MDN/Guides
- [2] MDN Web Docs, HTML guides: https://developer.mozilla.org/en-US/docs/Web/HTML/Guides
- [3] W3C, Web Content Accessibility Guidelines (WCAG) 2.2: https://www.w3.org/TR/WCAG22/